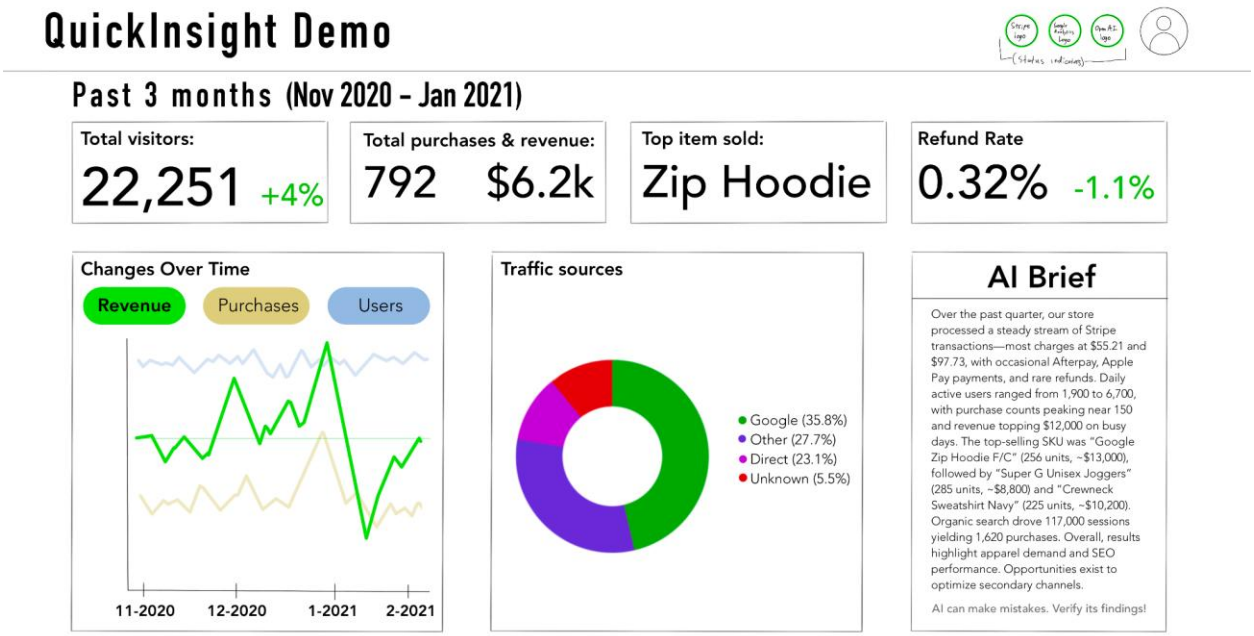
“QuickInsight” Project Log

July 2, 2025 - July 8th, 2025

Note: I started writing this log after 5 days of work on this project, so this first page is a record of what I recall doing over that period.

What I did:

- Created concept design image (see below)
- Set up project scaffolding with Vite, React, Material UI, and ESLint
- Attempted to use sandbox APIs for Google Analytics and Stripe to create test data; discovered limitations and dead ends. Used Google BigQuery’s public ecommerce dataset and adapted it to fit Stripe and Google Analytics’ API response styles instead.
- Recreated concept UI with React Material UI. Used placeholder text and data.
- Began adding internal API system to process and relay the mock data to the frontend.



Challenges:

- Overcoming design hurdles with React and Material UI. Thorough reading of the documentation ultimately solved the problem.

- Determining a sufficient data source to use as mock API responses from Stripe and Google Analytics. The documentation for the two API's served as my primary method of restructuring the data into the appropriate format

July 9, 2025

What I did:

- Refining the Stripe and Google Analytics API functions to get all the data they need for displaying up to a year's worth of data in one call each, rather than making multiple calls for different time frames.
- Added connections from the Routes folder programs to the main app.js file in the backend to test the retrieval of information from the mock files.

Challenges:

- Learning about the concept of Routers in Node, essentially delegating GET requests and their associated paths to their own separate files, rather than having them all in the main app file.
- Figuring out the best course of action to get the necessary data while minimizing the number of API calls. Ultimately decided that one call to retrieve 13 months of data and filtering it from there would be the most efficient.
- Ensuring that the API call format was consistent across all calls and between Stripe and Google Analytics.
- When updating the mock API calls to include the timeframe filtering, it would refuse to acknowledge the "timespan" filter in the URL. It took me a while to notice that I was not calling the API options through to the mock API call when I was for the live API version. Gotta love "simple" mistakes like that.

Next steps:

- Continue debugging the API system; still a few bugs left to fix.
- Update the frontend to include buttons for the “Past Year,” “Past 6 months,” “Past 3 months,” and “Past Month” time filters. Only the 3-month and 1-month buttons are available because that’s as far back as the test data goes.
- Connect the frontend to the backend to retrieve, display, and update the data when the page is loaded or when a timespan button is chosen.

July 10, 2025

What I did:

- Fixed bugs in the internal API system, made sure that time-filtering and rolling-window preferences were respected when processing and sending back the data.
- Added the buttons for filtering the time ranges, with dynamic disabling based on a value representative of how many months of data are available.

Challenges:

- When the API was returning filtered time ranges for the “changes over time” graph, it seemed that the first dates in the data were not on the first of the respective month (IE, it would start on November 3rd or January 2nd). I forgot to include logic to account for months of differing lengths. To keep things simple, I did a basic “30 days a month” value to calculate a rudimentary date (often landing on a date such as November 3rd), and then I would simply adjust it back to the first of that month.
- Unexpected behavior would occur if the API was called with no parameters (such as not specifying a filtering range), so I added a new middleware to enforce the period parameter and to return an error if a call was made without that parameter.

Next steps:

- Connect the backend to the frontend to display the mock API data.

July 11th, 2025

What I did:

- Migrated the MainContent.jsx file's code for the KPI card component back into its own Component class.
- Updated the time-range buttons to use React States to determine which buttons are available to select.
- Began connecting the frontend to the backend. Selecting time-range buttons updates the display on some of the buttons. Need to work on making them all work.
- Discovered and fixed a bug in the API, mostly pertaining to not respecting the time range.

Challenges:

- I did not know you needed to install CORS on NPM to allow your program to make fetch requests, which was necessary to do the API calls. That was relatively simple to set up.
- The Stripe API service was still using the old "1m", "3m", "6m," and "12m" markers for the period parameter, rather than the new version that omitted the letter "m." Quick fix, slipped right under my nose.

Next steps:

- Refine the KPI cards and then begin making the graphs

July 14th, 2025

What I did:

- Made changes to the frontend to improve readability on subvalues.
- Updated frontend to include dynamic truncation for both numbers and text alike. Used the Millify package on NPM to shorten large values.
- Installed the Recharts library to add the Changes Over Time graph.

Challenges:

- Creating a new React useEffect() to retrieve the changesOverTime data from the API. While the code for the KPI data was there, I wanted to see if I could do it from memory.
- Trying to find a way to display the multiple lines at once while displaying on proportionate scales rather than the same scale. Still trying to find a way to do that, ran out of time today.
- Styling the Graph buttons. For some reason, that was really finicky to get working.

Next steps:

- See if I can get the lines to all display together on proportion-based scales rather than absolute scales. No big deal if you can't, but it would be nice to have glancability at the other options even when something else is highlighted.
- Add the circle graph/pie chart for the traffic sources.

July 15th, 2025

What I did:

- Added many changes to the Changes Over Time graph.
    - Area graph for readability
    - Color and spacing adjustments
    - Date value is now visible in hover tooltip
    - Currency symbol in tooltip and y-axis ticks.
    - Y-axis values ran through nullify for readability.
- Updated the backend to share the currency symbol received from the data so the frontend can display the correct symbol. Used the NPM package "currency-symbol-map" for the conversion.
- Expanded the usage of the "millify" NPM package to perform on all numbers rather than checking if it's a monetary value first.
- Added Traffic Source pie chart with rolling number for the total users.

Challenges:

- Figuring out where/when to perform the currency symbol insertion. I originally had the backend do that, but then I realized that some currency codes equated to the same symbol, meaning there would be no way for the frontend to tell which currency it was (example: USD and ZWL both use the same dollar sign). So I updated the backend to share the currency symbol with the frontend, and let the frontend handle it instead.
- I originally attempted to adapt the old code that detected if the first character in a value/subvalue string was a dollar sign to decide whether or not to use millify. But I quickly realized that this was an odd restriction, and that I didn't need to adhere to this code structure. I redid the code to run millify on any number that was too long, rather than only run it on currency values. Much cleaner, much simpler.
- Struggling to figure out why the custom ChangesOverTime tooltip would refuse to render anything, only to realize that it was rendering white text on a white background…

Next steps:

- Figure out why the Total Visitors KPI card and the Total Users value in the Traffic Sources card have different values and sync them up.

- Begin working on ChatGPT integration, which will check if a summary has been made in the past hour and reuses that if it has, or uses OpenAI's API to generate and save a new summary.

July 16th, 2025

What I did:

- Figured out why the data doesn't line up between the total visitors KPI card and the total users in Traffic Sources. Ended up doing new queries in BigQuery's dataset and adapting them into the mock API responses, and found out the difference was due to the fact that the time series data showed UNIQUE visitors, while the traffic sources don't filter for unique users. I ended up changing the KPI card to specify that it was total unique users rather than visits.
- Made more queries to BigQuery's ga4_obfuscated_sample_ecommerce dataset to improve the mock data.
- Added rolling number animations for all fields.

Challenges:

- Querying BigQuery's ga4_obfuscated_sample_ecommerce dataset to fetch the traffic-source information CORRECTLY, with a query that correctly gets the daily traffic source breakdown, then formatting that data into the mock API format.

Next steps:

- Start adding the database to store the ChatGPT responses.
- Add the ChatGPT integration.

July 17, 2025

What I did:

- Set up a PostgreSQL database for caching API responses to reduce the number of API calls needed and thus decrease costs (if there were any).
- Consolidated the API system to just be one file that also handles caching the received data.
- Added animation to text-only KPI values/subvalues if the number rolling animation does not apply to the text space.

Challenges:

- Figuring out the most effective data types for the columns I would need in PostgreSQL.
- Rewriting the stripeService and gaService files into a new, streamlined “APIService.js” file to work with this new caching system.

Next steps:

- Test that the “write to cache on new hour or older than an hour” rule works. Writing on the next hour, I just need to make sure that beyond an hour works. Check this first thing in the morning.
- Add the OpenAI integration and engineer a prompt using the data as part of the information it receives. Do one for each viewable range based on the data and cache each response.

July 18th, 2025

What I did:

- Verified that the caching system updates in both intended cases. It does.
- Set up a new OpenAI account specifically for API usage.
- Added new columns to the database to store the store name and store industry for the AI prompt.

Challenges:

- Figuring out OpenAI’s API and crafting a good prompt.
- Fixing a bug where it would do all API calls twice. The source was React.StrictMode in the frontend. Apparently it calls everything twice.
- Trying to fix a bug where changing the period would the API again if USE_CACHE_BYPASS is set to true.

Next steps:

- Fix the bug with repeated API calls when they are not necessary.

July 21st, 2025

What I did:

- Fixed a bug where USE_CACHE_BYPASS being set to true would cause new data to be requested every time a timeframe button was pressed.
- Added a “business name” entry at the top of the UI.
- Fixed light/dark mode theming issue
- Added text animation to AI Brief section

Challenges:

- The reason for the bug was that when I was adding the bypass functionality, I pasted it into a function that was called each time a button was pressed instead of a page refresh.

Next steps:

- Polish the User Interface (colors, light/dark adaptivity, different screen widths, themes, etc)
- Make any other changes I can think of before proceeding to AWS segment of the project

August 5th, 2025

What I did:

- After a prolonged break from QuickInsight to complete my AWS Certification exam, I returned to start uploading the project to AWS to prove my knowledge.
- Created an S3 Bucket to store the built website statically (made with “npm run build” and syncing via command line)
- Created a CloudFront distribution to deliver the contents of S3 using Origin Access Control.
- Updated the frontend to display an error message if the backend could not be contacted.
- Researched available domain names in AWS Route 53 for the website: “quickinsightdemo.io” was $71 a year, so I changed it to .com for $15 a year.
- Began creating an Elastic Beanstalk application for the Backend, uploaded via a cleaned ZIP file (no dotenv secrets file or node_modules folder), and the appropriate IAM service roles for it.
- Created RDS PostgreSQL database to connect to the backend, with the password handled by AWS Secrets Manager. Copied the local DB into RDS as well.
- Updated the backend code to call the RDS database instead of the local database

Challenges:

- Elastic Beanstalk has severe health check problems immediately every time I upload it. Some changes involved changing the Express server port to one that Elastic Beanstalk Node.js servers expect, and a healthcheck endpoint. Still not working, though

Next steps:

- Get Elastic Beanstalk working properly
- Update the frontend in S3 to call the EB API

August 6th, 2025

What I did:

- Continued trying to get the backend running on Elastic Beanstalk
- Trying to use EB CLI tool provided by Amazon to get the backend posted to EB.
- Still not working.

Challenges:

- EB CLI tool refused to work and I had to reinstall Python to use it. I also had to do a lot of PATH Environment Variable editing and running Admin Powershell.
- Environment Variables appear to be having issues still, according to the logs.
- Thought I had to move EB into the relevant subnets to allow communication with my RDS database. Still didn't seem to resolve the issue.

Next steps:

- Get Elastic Beanstalk working, still.
- Update frontend to call API at EB once that is running.

August 7th, 2025

What I did:

- Fixed last of the Elastic Beanstalk deployment issues for backend
- Adjusted the API endpoint for frontend and reuploaded it to S3/CloudFront
- Registered “quickinsightdemo.com” as a Route 53 domain name
- Got SSL certificates for quickinsightdemo.com and its subdomains.

Challenges:

- Learned that setting an environment variable to use Secrets Manager will automatically get the secret for you when in EB. I don’t need to extract it myself. I had to add some additional logic to use Secrets Manager decoding on local environment, and use the raw value for AWS deployment
- CloudFront-deployed frontend failed to contact backend due to an HTTPS / HTTP difference. I need to get a domain and SSL certificate for the backend.

Next steps:

- Get SSL certs attached to the domain, Cloudfront, and EB
- Attach the registered domain to the frontend CloudFront

August 8th, 2025

What I did:

- Got the SSL certificates for the domain registered and validated.
- Attached backend to an API Route 53 alias, updated frontend API endpoint to use the Route 53 endpoint.
- Attached the frontend to the Route 53 domain name.

Challenges:

- None, today was smooth sailing.

Next steps:

- All done!

August 9th, 2025

What I did:

- I forgot to make a mobile version! I spent today doing that and redesigning the interface for different screen widths.
- Added more colors to the interface in both light and dark mode.

Challenges:

- Kept having issues with the time range buttons getting broken with every change I made. Keeping it visually consistent while being adaptive to screen size was challenging.

Next steps:

- Now I am done!